

Optimización de CNN de clasificación con herramientas HPC

Andrés Mardones Domcke

andres.mardones@alumnos.uach.cl

Abstract

Este trabajo compara cuatro implementaciones de una red neuronal convolucional de clasificación utilizando distintas técnicas de optimización de alto rendimiento: Python base, vectorización con NumPy, paralelización en CPU con Cython y ejecución en GPU mediante JAX. Se utilizó CIFAR-10 y se evaluaron tiempos de convolución, forward y entrenamiento completo. Los resultados muestran que NumPy obtuvo la mayor aceleración con operaciones vectorizables, mientras que JAX presentó el mejor desempeño global en GPU para entrenamiento completo entre las técnicas paralelas. El trabajo confirma el impacto de técnicas HPC en reducción de tiempo para CNN.

1. Introducción

El reciente crecimiento de la visión por computador ha sido muy relevante en la industria de la informática y la inteligencia artificial. Existen muchas aplicaciones que aportan soluciones en diversas áreas, tales como medicina, automatización industrial, deporte, astronomía, entre otras. Dentro de esta rama existe un apartado dedicado exclusivamente a las redes neuronales, técnica por excelencia en el estado del arte para dar solución a tareas como extracción de características, detección, tracking, segmentación o clasificación.

Sin embargo, el éxito de las redes neuronales convolucionales trae consigo un claro problema: el coste computacional. Y es que muchos aspectos inherentes al funcionamiento de las redes provocan que el procesamiento sea muy grande. Por ejemplo, la arquitectura de la red, es decir, la cantidad de filtros, de capas (profundidad), o el tamaño de las imágenes a procesar. Al aumentar estos parámetros, los tiempos de ejecución pueden aumentar considerablemente. Además, para obtener buenos resultados, ya sea para aumentar la precisión de la red o para evitar el sobreajuste, se puede requerir de aumentar el tamaño del dataset o de entrenar por más épocas, incrementando el coste computacional.

Es por estas razones que resulta lógico buscar soluciones relacionadas a optimizar las operaciones internas de las redes neuronales convolucionales con tal de atenuar el cre-

cimiento de los tiempos de ejecución a medida que las redes escalan en alguno de sus parámetros. Y es aquí donde cobra importancia la computación de alto rendimiento, un área que ofrece herramientas pensadas para reducir los tiempos de ejecución y utilizar los recursos de manera eficiente en problemas complejos.

El objetivo de este informe es comparar y analizar de forma justa 4 implementaciones de una red neuronal convolucional, con el fin de obtener conclusiones acerca de cuál enfoque puede resultar más adecuado dada la naturaleza de los cálculos que se realizan en un problema como este. Esas 4 implementaciones son:

- Versión con operaciones base de Python
- Versión mejorada utilizando vectorización con NumPy
- Versión paralela en CPU usando Cython
- Versión paralela en GPU usando JAX.

A continuación se procede a explicar la naturaleza del problema, así como la implementación de las versiones y los resultados de las comparaciones entre ellas.

2. Estudio del rendimiento

En este apartado se presenta un análisis del coste computacional de las principales operaciones que participan en una red neuronal convolucional.

2.1. Convolución

La convolución corresponde a la operación central dentro de una red neuronal convolucional. Su objetivo es extraer patrones locales de la imagen de entrada mediante la aplicación de filtros o *kernels*. Cada filtro recorre espacialmente la imagen y, en cada posición, realiza un producto punto entre los valores del kernel y la región correspondiente de la entrada, generando como resultado un mapa de activación.

Desde un punto de vista matemático, para una entrada bidimensional (x) y un kernel (w) de tamaño $(K \times K)$, la salida puede expresarse como:

$$y(i, j) = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} x(i + m, j + n) w(m, n) \quad (1)$$

Esta operación debe repetirse para cada posición espacial de la imagen, para cada canal de entrada y para cada filtro definido en la capa. Como consecuencia, la cantidad de operaciones crece rápidamente a medida que aumenta el tamaño del problema. Considerando una entrada de dimensiones $(H \times W)$, (C) canales y (F) filtros, el costo computacional de una convolución puede aproximarse por:

$$\mathcal{O}(H \cdot W \cdot C \cdot K^2 \cdot F) \quad (2)$$

Desde la perspectiva de implementación, la convolución representa un cuello de botella debido a que involucra múltiples bucles anidados y una gran cantidad de accesos a memoria. En una implementación base utilizando Python, estas operaciones se realizan mediante ciclos *for* explícitos, lo que introduce un alto costo. Este comportamiento convierte a la convolución en el principal candidato para aplicar optimizaciones de alto rendimiento. Además, la naturaleza de esta operación presenta un alto grado de paralelismo: distintas posiciones espaciales y distintos filtros pueden calcularse de forma independiente. Esto permite aprovechar técnicas como vectorización con NumPy, paralelización en CPU o aceleración en GPU.

2.2. ReLU

La función de activación ReLU (*Rectified Linear Unit*) se aplica luego de la convolución con el objetivo de introducir no linealidad en la red. Esto permite que la CNN pueda aprender representaciones más complejas que una combinación lineal de filtros.

Su definición matemática es:

$$f(x) = \max(0, x) \quad (3)$$

Desde el punto de vista computacional, ReLU tiene un costo lineal respecto a la cantidad de elementos procesados:

$$\mathcal{O}(H \cdot W \cdot C) \quad (4)$$

A diferencia de la convolución, esta operación no requiere multiplicaciones ni acumulaciones, por lo que su costo individual es bajo. Sin embargo, al aplicarse sobre todos los mapas de activación de cada capa, sigue representando una parte relevante del tiempo total de ejecución.

Además, debido a que cada elemento puede evaluarse de forma independiente, ReLU presenta un alto nivel de paralelismo, siendo una operación especialmente favorable para vectorización y ejecución en GPU.

2.3. Pooling

Luego de las capas convolucionales, es común aplicar una operación de *pooling* con el objetivo de reducir la dimensionalidad espacial de los mapas de activación. Esta operación resume la información contenida en regiones pequeñas de la entrada, conservando únicamente los valores más representativos. Entre las variantes más utilizadas destacan *max pooling*, que selecciona el valor máximo dentro de una ventana, y *average pooling*, que calcula el promedio de los elementos presentes en dicha región.

Desde el punto de vista computacional, el costo aproximado del *pooling* puede expresarse como:

$$\mathcal{O}(H \cdot W \cdot C) \quad (5)$$

donde (H) y (W) representan las dimensiones espaciales y (C) la cantidad de canales.

Al igual que en la convolución, el cálculo de cada ventana puede realizarse de manera independiente, lo que permite aprovechar paralelismo a nivel de datos.

2.4. Capa Fully Connected

La capa *fully connected* corresponde a la etapa final de la red neuronal y se encarga de combinar las características extraídas por las capas anteriores para realizar la clasificación.

En esta capa, cada neurona se conecta con todas las entradas recibidas, realizando una combinación lineal entre el vector de entrada y los pesos asociados. Su expresión puede representarse como:

$$y = Wx + b \quad (6)$$

donde (x) corresponde al vector de entrada, (W) a la matriz de pesos y (b) al sesgo de la capa.

Desde el punto de vista computacional, el costo aproximado está dado por:

$$\mathcal{O}(M \cdot N) \quad (7)$$

donde (M) representa la cantidad de entradas y (N) la cantidad de neuronas de salida.

Aunque esta operación suele tener un costo menor que las capas convolucionales, puede representar una parte importante del tiempo de ejecución cuando el vector de entrada es grande. Además, al tratarse principalmente de multiplicaciones matriciales, resulta especialmente favorable para vectorización y paralelización tanto en CPU como en GPU.

2.5. Softmax

La función *Softmax* se utiliza en la capa de salida de la red para transformar los valores obtenidos en probabilidades asociadas a cada clase.

Su definición matemática es:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}} \quad (8)$$

donde (N) corresponde al número de clases.

Desde el punto de vista computacional, esta operación tiene complejidad lineal respecto a la cantidad de clases:

$$\mathcal{O}(N) \quad (9)$$

Su costo es considerablemente menor, ya que se ejecuta únicamente sobre el vector final de salida. Sin embargo, resulta fundamental dentro de la red al permitir interpretar la salida como una distribución de probabilidades y realizar la clasificación final.

2.6. Backward Propagation

Además de la etapa de forward, el entrenamiento de una red neuronal convolucional requiere ejecutar una etapa de retropropagación o *backward propagation*. Su objetivo es calcular cómo afecta cada parámetro de la red al valor de la función de pérdida y, a partir de ello, actualizar pesos y sesgos durante el entrenamiento. En términos generales, la retropropagación aplica la regla de la cadena para propagar el gradiente desde la salida de la red hacia las capas anteriores. De esta forma, cada capa debe calcular gradientes respecto a su salida, sus entradas y sus parámetros.

Desde el punto de vista computacional, esta etapa suele representar uno de los mayores costos dentro del entrenamiento, debido a que requiere recorrer nuevamente todas las capas de la red y realizar múltiples operaciones adicionales sobre los datos ya procesados durante la etapa *forward*. En particular, las capas convolucionales resultan especialmente costosas, ya que el cálculo de gradientes involucra una cantidad elevada de multiplicaciones y accesos a memoria. Desde el punto de vista computacional, el costo de esta etapa puede aproximarse por:

$$\mathcal{O}(H \cdot W \cdot C \cdot K^2 \cdot F) \quad (10)$$

donde (H) y (W) representan las dimensiones espaciales, (C) la cantidad de canales, (K) el tamaño del kernel y (F) la cantidad de filtros. El orden de complejidad es similar al de la convolución en la etapa *forward*, aunque en la práctica suele presentar un costo mayor debido al cálculo simultáneo de múltiples gradientes y al aumento en accesos a memoria. A pesar de este costo, la backward propagation presenta un alto nivel de paralelismo. Muchos de los gradientes asociados a filtros, mapas de activación y muestras del *batch* pueden calcularse de forma independiente, permitiendo aplicar estrategias de vectorización y paralelización tanto en CPU como en GPU.

3. Implementación

Para comparar los diferentes métodos se eligió una arquitectura de red neuronal convolucional de clasificación de imágenes con las siguientes características:

- 1 capa convolucional con 8 filtros 3x3x3
- Función de activación ReLU
- Capa de reducción MaxPooling 2x2
- 1 capa fully connected con 10 neuronas
- Softmax

El dataset a utilizar es CIFAR-10, el cual presenta unas 60000 imágenes de resolución 32x32 distribuidas en 10 clases.

3.1. Versión Python base

Esta versión se incluye con el fin de analizar el rendimiento de la implementación más básica, estableciendo el piso sobre el cual mejorar. Para eso se decide utilizar Python puro, lo que implica ejecutar todo de manera secuencial, anidando bucles for para las operaciones incluidas en la red.

Algorithm 1 Convolución en Python puro

```

1: Entrada: Imagen  $X \in \mathbb{R}^{H \times W}$ , filtro  $W \in \mathbb{R}^{K \times K}$ 
2: Salida: Mapa de activación  $Y$ 
3:  $output\_height \leftarrow H - K + 1$ 
4:  $output\_width \leftarrow W - K + 1$ 
5: Inicializar  $Y$  con ceros
6: for  $i = 0$  to  $output\_height - 1$  do
7:   for  $j = 0$  to  $output\_width - 1$  do
8:      $suma \leftarrow 0$ 
9:     for  $m = 0$  to  $K - 1$  do
10:      for  $n = 0$  to  $K - 1$  do
11:         $suma \leftarrow suma + X[i + m][j + n] \cdot$ 
            $W[m][n]$ 
12:      end for
13:    end for
14:     $Y[i][j] \leftarrow suma$ 
15:  end for
16: end for
17: return  $Y$ 

```

En este pseudocódigo de la convolución con Python puro se aprecia el coste computacional del algoritmo como tal, forzando a incluir 4 bucles for anidados (se ignoraron canales de entrada y múltiples filtros, incluidos en la implementación real), además accediendo constantemente a memoria para calcular las sumas y multiplicaciones. Este patrón se repite constantemente en las demás partes de la

red, puesto que tanto ReLU, como MaxPooling, SoftMax y la capa Fully Connected necesitan recorrer arreglos.

3.2. Versión vectorizada con NumPy

La segunda versión implementada corresponde a mejoras con respecto a la vectorización de las operaciones. A diferencia de Python puro, donde gran parte de las operaciones se ejecutan mediante ciclos explícitos interpretados uno a uno, NumPy agrupa múltiples cálculos sobre arreglos completos y los delega a rutinas internas implementadas en lenguaje compilado. Esto disminuye el costo asociado al intérprete de Python y aprovecha mejor la memoria y el procesador. Además, muchas de estas operaciones internas se encuentran optimizadas para trabajar sobre bloques contiguos de datos y utilizar instrucciones vectoriales del hardware. Como resultado, se mantiene la misma lógica matemática de la red neuronal, pero con una ejecución considerablemente más eficiente.

Un ejemplo de esta mejora viene con la convolución, donde el procesamiento se reduce notablemente dada la condición vectorial de la operación:

Algorithm 2 Convolución vectorizada con NumPy

```
1: Entrada: Imagen  $X$ , filtros  $W$ 
2: Salida: Mapa de activación  $Y$ 
3:  $windows = sliding\_window\_view(X, (3, 3))$ 
4:  $windows = windows.transpose(0, 1, 3, 4, 2)$ 
5:  $Y = np.tensordot(windows, W)$ 
6: return  $Y$ 
```

En el código de esta versión para la convolución no se ve ningún bucle for, todas las operaciones se producen de forma vectorizada. Comenzando por `sliding_window_view()`, una función que genera todas las ventanas de la imagen para aplicarles convolución. *Transpose*, reorganiza los ejes y *tensordot* es la encargada de multiplicar las ventanas con los filtros de forma simultánea. Todas estas funciones, al pertenecer a la librería de NumPy, cuentan con una optimización elevada, principalmente, debido a que están construidas en C. Cabe mencionar que todas las demás etapas de la red fueron también optimizadas con funciones de NumPy, utilizando operaciones como `np.max()` o `np.sum()`

3.3. Versión paralelizada en CPU con Cython

Si bien existen muchas formas de paralelizar una red neuronal convolucional, se decidió por aplicarla solo en la convolución, sobre la dimensión de filtros, distribuyendo el cálculo de cada filtro entre distintos hilos mediante la instrucción `prange`. De esta manera, cada hilo ejecuta de forma independiente la convolución correspondiente sobre toda la imagen para un subconjunto de filtros, calculando en paralelo sus mapas de activación. Además del paralelismo,

se utilizó tipado estático y vistas de memoria en Cython, reduciendo el costo asociado al intérprete de Python y mejorando el acceso a memoria. Debido a que el cálculo de cada filtro es independiente del resto, esta estrategia permite aprovechar eficientemente la arquitectura multicore de CPU y reducir el tiempo total de ejecución de la capa convolucional.

Algorithm 3 Convolución paralela en CPU con Cython

```
1: Entrada: Imagen  $X$ , filtros  $W$ 
2: Salida: Mapa de activación  $Y$ 
3: Inicializar  $Y$ 
4: for en paralelo cada filtro  $f$  do
5:   for cada posición  $(i, j)$  de la imagen do
6:      $Y[i, j, f] \leftarrow convolve\_region(X, W, f, i, j)$ 
7:   end for
8: end for
9: return  $Y$ 
```

Dentro de la versión paralela en CPU, la optimización se concentró únicamente en la capa convolucional. Esta decisión se tomó debido a que, según el análisis teórico y las pruebas iniciales, la convolución representa la operación de mayor costo computacional dentro de la CNN, concentrando la mayor parte del tiempo de ejecución. En comparación, etapas como ReLU, *pooling*, capa *fully connected* y *softmax* presentan una menor carga computacional relativa y, por tanto, su paralelización entrega un beneficio más reducido frente al costo adicional de implementación y sincronización.

De esta forma, se priorizó optimizar el principal cuello de botella del sistema. Sin embargo, etapas como *pooling* y *backward propagation* también presentan oportunidades de paralelización, las cuales pueden considerarse como trabajo futuro.

3.4. Versión paralelizada en GPU con JAX

La implementación paralela en GPU fue desarrollada utilizando JAX, biblioteca orientada a cómputo numérico acelerado que permite ejecutar operaciones sobre GPU mediante compilación optimizada. La convolución fue encapsulada dentro de una función decorada con `@jax.jit`, lo que permite que JAX compile la operación completa antes de su ejecución y genere una versión optimizada adaptada al hardware disponible.

A diferencia de la versión CPU, donde la paralelización fue aplicada explícitamente sobre la dimensión de filtros, en GPU el paralelismo es gestionado automáticamente por JAX. La operación completa de convolución se transforma en un kernel ejecutado en paralelo sobre múltiples posiciones espaciales y filtros de manera simultánea.

Durante *forward*, la imagen de entrada se convierte a un arreglo de JAX y luego se extraen

todas las ventanas de tamaño (3×3) mediante `conv_general_dilated_patches`. Posteriormente estas ventanas se reorganizan para coincidir con la estructura de los filtros y finalmente la convolución se realiza mediante `tensordot`.

4. Resultados

La experimentación se centra en los tiempos de ejecución presentados por cada una de las versiones, midiendo tanto la convolución, el forward completo y el train completo. Dentro de la metodología de la experimentación, se encuentran las siguientes características:

- Dataset: CIFAR-10
- 500 imágenes de tamaño $32 \times 32 \times 3$
- 10 repeticiones por benchmark
- Tipo de dato float32 para todas las versiones
- Benchmarks: tiempo de convolución, tiempo de forward completo y tiempo de train completo.

Table 1. Resultados de rendimiento promedio para 500 imágenes y 10 repeticiones.

Versión	Benchmark	Promedio (s)	Desv. Est.	Speedup
Python	Conv forward	53.155	0.508	1.00
NumPy	Conv forward	0.051	0.012	1048.43
Cython CPU	Conv forward	0.136	0.016	390.73
JAX GPU	Conv forward	1.069	0.107	49.73
Python	Forward red	58.087	0.417	1.00
NumPy	Forward red	0.100	0.016	580.98
Cython CPU	Forward red	6.379	0.249	9.11
JAX GPU	Forward red	2.070	0.188	28.07
Python	Train completo	134.522	0.759	1.00
NumPy	Train completo	0.288	0.032	466.64
Cython CPU	Train completo	19.151	0.173	7.02
JAX GPU	Train completo	12.689	1.027	10.60

5. Conclusiones

Los resultados obtenidos muestran que la implementación base en Python presentó el peor desempeño en todos los benchmarks. Esto era esperable, ya que la red utiliza múltiples bucles anidados en las etapas de convolución, pooling y fully connected, generando un alto overhead del intérprete y un bajo aprovechamiento del hardware disponible. Esta versión fue utilizada como línea base para medir las aceleraciones obtenidas en el resto de las implementaciones.

La vectorización con NumPy fue la optimización que entregó la mayor mejora de rendimiento. En la convolución se observó una aceleración superior a 1000 veces respecto

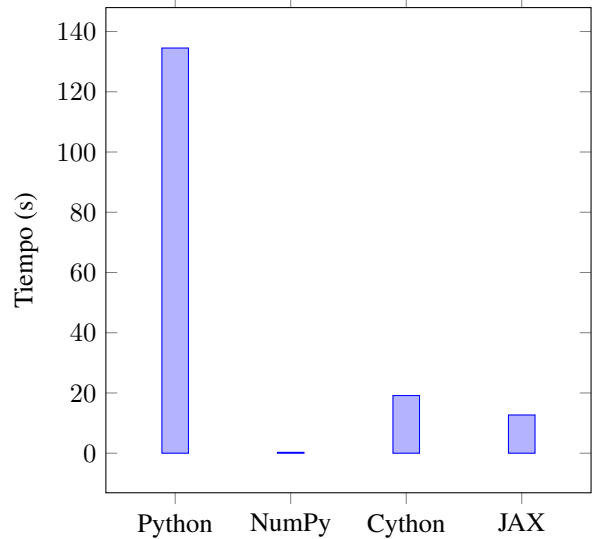


Figure 1. Tiempo promedio de entrenamiento completo para 500 imágenes.

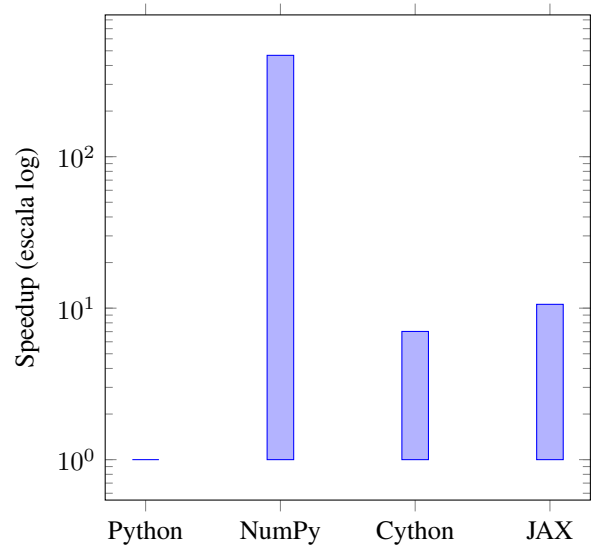


Figure 2. Speedup respecto a la implementación base.

a Python base, mientras que en el entrenamiento completo el speedup fue cercano a 466 veces. Este comportamiento se explica principalmente por la eliminación de los bucles explícitos en Python y el uso de operaciones vectorizadas implementadas en C altamente optimizadas, como `tensordot`, `outer`, `reshape` y operaciones matriciales internas. Los resultados muestran que, para este tamaño de red y de imágenes, la vectorización fue la técnica más efectiva.

La implementación paralela en CPU mediante Cython y OpenMP logró mejorar significativamente el tiempo respecto a Python base, especialmente en la capa de convolución, donde se paralelizó el cálculo entre filtros uti-

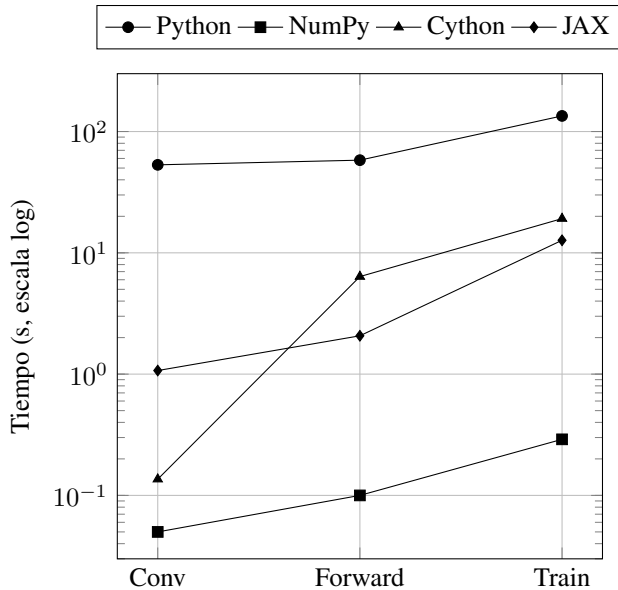


Figure 3. Comparación de tiempos promedio por etapa.

lizando múltiples hilos. En entrenamiento completo se obtuvo un speedup cercano a 7 veces. Aunque el rendimiento fue considerablemente mejor que Python puro, no logró superar a NumPy. Esto sugiere que, para un problema de esta escala, el costo de coordinación entre hilos y el acceso a memoria reduce parte del beneficio del paralelismo explícito, evidenciado claramente en la figura, donde el tiempo de la convolución como tal es bajo (el segundo más bajo), pero aumenta excesivamente en el forward y train. Esto podría indicar que la forma de paralelizar la red neuronal, por filtros, podría no haber aprovechado completamente el paralelismo disponible en CPU. Durante la experimentación de la versión paralela en CPU se utilizaron 6 hilos de ejecución (threads) mediante OpenMP, buscando aprovechar los núcleos disponibles del procesador y distribuir el trabajo de la capa de convolución entre múltiples tareas concurrentes. En esta implementación, cada hilo procesa distintos filtros de manera independiente, por lo que el rendimiento está directamente relacionado con la cantidad de filtros utilizada. Cuando el número de filtros es mayor, existe más trabajo que repartir entre los hilos y el paralelismo puede aprovecharse de mejor forma, reduciendo el tiempo total de ejecución. En cambio, con una cantidad menor de filtros o con cargas de trabajo pequeñas, el beneficio puede disminuir debido al costo adicional asociado a la creación, sincronización y coordinación entre threads, lo que impacta directamente en los tiempos medidos.

La versión implementada en GPU mediante JAX obtuvo los mejores resultados entre las alternativas paralelas en entrenamiento completo, con una aceleración aproximada de

10.6 veces respecto a Python base. Sin embargo, en la convolución aislada NumPy fue más rápido que JAX. Esto puede explicarse porque la GPU entrega mejores resultados cuando el costo computacional es suficientemente grande para amortizar el overhead de compilación JIT, transferencia de datos y ejecución en el dispositivo. En operaciones pequeñas o individuales, ese overhead puede ser mayor que el beneficio de la paralelización masiva.

En términos generales, los resultados permiten concluir que la técnica óptima depende del tipo de carga de trabajo. Para operaciones densas y altamente vectorizables, NumPy entregó el mejor desempeño. Para aprovechar CPU multinúcleo, Cython permitió implementar paralelismo explícito y obtener mejoras importantes. Para cargas más completas y con mayor volumen de cálculo, JAX y GPU mostraron un mejor aprovechamiento del paralelismo del hardware. Esto confirma que la computación de alto rendimiento aplicada a redes convolucionales puede reducir de forma importante los tiempos de ejecución, pero que el beneficio depende de la naturaleza del problema y del costo asociado a cada estrategia de optimización.

Como trabajo futuro sería interesante ampliar el análisis utilizando una arquitectura de mayor tamaño, aumentar el número de filtros y capas convolucionales, medir escalabilidad con distintos números de hilos, aplicar otras formas de paralelizar (por batches o por fila de la imagen), o comparar el rendimiento con librerías especializadas como PyTorch o TensorFlow. Esto permitiría evaluar con mayor profundidad el comportamiento de cada estrategia en escenarios más cercanos a aplicaciones reales de deep learning.

6. Anexo

Todo el detalle de la implementación se encuentra en el repositorio de GitHub: <https://github.com/AndresMardonesUACH/CNN-optimization-with-HPC-tools->